Projet : Parking Intelligent avec ESP32 et ESP32-CAM

1. Objectif du projet

Réaliser un système de parking intelligent capable de : - Détecter la disponibilité de **3 places de parking** via capteurs ultrasoniques. - Gérer une **barrière automatique** à l'aide d'un servomoteur. - Autoriser l'accès soit par **reconnaissance de plaque** (ESP32-CAM + base de données), soit par **RFID**. - Afficher des messages (*Bienvenue*, *Accès refusé*, *Places disponibles*) sur un **écran OLED**.

---

2. Matériel utilisé

- ESP32 (carte principale)
- ESP32-CAM + adaptateur USB
- 5 capteurs ultrasoniques HC-SR04
- 1 servomoteur SG90
- 1 module RFID RC522 (I2C : VCC, GND, SDA, SCL)
- 2 cartes RFID (IDs enregistrés)
- Écran OLED SSD1306 (I2C)
- Fils de connexion

---

3. Architecture du système

Répartition des capteurs ultrasoniques

- US1, US2, US3 : disponibilité des places
- US4 : détection voiture **avant la barrière**
- US5 : détection voiture **après la barrière**

Modes d'accès

1. **RFID valide** → ouverture directe de la barrière
2. **Plaque reconnue** → ouverture
3. Sinon → accès refusé

---

4. Câblage complet et détaillé

4.1 ESP32 (carte principale)

Capteurs ultrasoniques HC-SR04

Fonction	TRIG	ECHO
Place 1	GPIO 13	GPIO 12
Place 2	GPIO 14	GPIO 27
Place 3	GPIO 26	GPIO 25
Avant barrière	GPIO 33	GPIO 32
Après barrière	GPIO 18	GPIO 19

- VCC → 5V
- GND → GND

⚠️Astuce : utiliser un diviseur de tension sur ECHO (5V → 3.3V)

Servomoteur (SG90)

- Signal → GPIO 4
- VCC → 5V externe
- GND → GND commun

Écran OLED SSD1306 (I2C)

- SDA → GPIO 21
- SCL → GPIO 22
- VCC → 3.3V
- GND → GND

RFID RC522 (I2C – 4 fils)

- SDA → GPIO 21
- SCL → GPIO 22
- VCC → 3.3V
- GND → GND

4.2 ESP32-CAM

- Alimentation : 5V (adaptateur USB)
- Programmation : via adaptateur FTDI
- Communication : WiFi (HTTP)

------|------|------| | US1 | GPIO 13 | GPIO 12 | | US2 | GPIO 14 | GPIO 27 | | US3 | GPIO 26 | GPIO 25 | | US4 | GPIO 33 | GPIO 32 | | US5 | GPIO 18 | GPIO 19 | VCC → 5V, GND → GND

4.2 Servomoteur

- Signal → GPIO 4
- VCC → 5V
- GND → GND

4.3 OLED SSD1306 (I2C)

- SDA → GPIO 21
- SCL → GPIO 22
- VCC → 3.3V
- GND → GND

4.4 RFID RC522 (I2C)

- SDA → GPIO 21
- SCL → GPIO 22
- VCC → 3.3V
- GND → GND

4.5 ESP32-CAM

- Alimentée séparément via adaptateur USB
- Communication via WiFi (requêtes HTTP)

---

5. Principe de fonctionnement

1. Les capteurs ultrasoniques détectent les places libres.
2. L'OLED affiche le nombre de places disponibles.
3. Une voiture arrive devant la barrière.
4. Le système vérifie :
5. Carte RFID valide ? → ouvrir
6. Sinon : demande à l'ESP32-CAM la reconnaissance de plaque
7. Si autorisé → servomoteur ouvre la barrière.
8. US5 confirme que la voiture est passée → barrière se referme.

---

6. Code ESP32 principal

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <Servo.h>
#include <MFRC522_I2C.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

Servo barriere;

MFRC522_I2C rfid(0x28);

String carte1 = "A1 B2 C3 D4";
String carte2 = "11 22 33 44";

int servoPin = 4;

long lireDistance(int trig, int echo) {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long duree = pulseIn(echo, HIGH);
  return duree * 0.034 / 2;
}

void ouvrirBarriere() {
  barriere.write(90);
  delay(3000);
  barriere.write(0);
}

void setup() {
  Serial.begin(115200);
  Wire.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();

  barriere.attach(servoPin);
  barriere.write(0);

  rfid.begin();
}

void loop() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);

  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
    String id = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
      id += String(rfid.uid.uidByte[i], HEX) + " ";
    }
    id.toUpperCase();
```

```
    if (id == carte1 || id == carte2) {
      display.println("Bienvenue");
      display.display();
      ouvrirBarriere();
    } else {
      display.println("Acces refuse");
      display.display();
    }
  }
  delay(500);
}
```

---

7. ESP32-CAM + Serveur Python (détaillé)

7.1 Rôle de l'ESP32-CAM

- Capturer l'image de la voiture devant la barrière
- Envoyer l'image au serveur Python via WiFi (HTTP POST)
- Recevoir la réponse du serveur : **AUTORISE** ou **REFUSE**

7.2 Code ESP32-CAM (Arduino IDE)

```
#include "esp_camera.h"
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "WIFI_NAME";
const char* password = "WIFI_PASSWORD";
const char* serverUrl = "http://192.168.1.100:5000/upload";

void setup() {
  Serial.begin(115200);

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = 5;
  config.pin_d1 = 18;
  config.pin_d2 = 19;
  config.pin_d3 = 21;
  config.pin_d4 = 36;
  config.pin_d5 = 39;
  config.pin_d6 = 34;
  config.pin_d7 = 35;
  config.pin_xclk = 0;
  config.pin_pclk = 22;
  config.pin_vsync = 25;
```

```
  config.pin_href = 23;
  config.pin_sscb_sda = 26;
  config.pin_sscb_scl = 27;
  config.pin_pwdn = 32;
  config.pin_reset = -1;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;
  config.frame_size = FRAMESIZE_VGA;
  config.jpeg_quality = 12;
  config.fb_count = 1;

  esp_camera_init(&config);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void loop() {
  camera_fb_t * fb = esp_camera_fb_get();
  if (!fb) return;

  HTTPClient http;
  http.begin(serverUrl);
  http.addHeader("Content-Type", "image/jpeg");
  int httpResponseCode = http.POST(fb->buf, fb->len);

  if (httpResponseCode == 200) {
    String response = http.getString();
    Serial.println(response); // AUTORISE ou REFUSE
  }

  http.end();
  esp_camera_fb_return(fb);
  delay(5000);
}
```

7.3 Serveur Python (Flask + reconnaissance plaque)

```
from flask import Flask, request
import cv2
import pytesseract
import numpy as np

app = Flask(__name__)

plaques_autorisees = ["123TN456", "987TN321"]
```

```
@app.route('/upload', methods=['POST'])
def upload():
    image = np.frombuffer(request.data, np.uint8)
    img = cv2.imdecode(image, cv2.IMREAD_COLOR)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    texte = pytesseract.image_to_string(gray)

    for plaque in plaques_autorisees:
        if plaque in texte:
            return "AUTORISE"
    return "REFUSE"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

7.4 Communication ESP32 ↔ ESP32-CAM

- L'ESP32 principal attend la réponse **AUTORISE**
- Si OK → ouverture barrière
- Sinon → affichage *Accès refusé*

---

8. Résultat attendu

- Affichage clair sur OLED
- Barrière automatique fiable
- Double sécurité : RFID + plaque
- Système évolutif (application mobile, cloud)

---

9. Conclusion

Ce projet de parking intelligent combine efficacement : - L'IoT (ESP32, capteurs, RFID) - La vision par ordinateur (ESP32-CAM + OCR) - L'automatisation (barrière intelligente)

Il offre une solution fiable, sécurisée et évolutive pour la gestion des parkings modernes.

---

10. Améliorations possibles

- Application mobile
- Base de données cloud
- Statistiques d'occupation
- Reconnaissance faciale

---

11. Schéma global (logique)

Voiture → Ultrason → RFID / Caméra → Serveur → ESP32 → Servo → OLED